Verdict-Optimized Trace Abstraction: Decision-Relevant Trace Compression for Test Outcome Prediction

Anonymous

Abstract

Execution trace SFT has shown limited benefit for code generation tasks, but its effectiveness on code *verdict* tasks—binary pass/fail prediction—remains unexplored. We introduce Verdict-Optimized Trace Abstraction (VOTA), a compact trace representation that retains only decision-relevant execution states: exception summaries, loop compression markers, and assertion structure. On test outcome prediction (n=1,190), VOTA achieves 99.2% accuracy, outperforming both no-trace (75.1%) and full-trace SFT (81.8%) by +24.0 pp and +17.4 pp respectively. Component ablation reveals that exception summarization is the single most important design choice, contributing ∼83% of VOTA's total gain, while loop and assertion components provide marginal but consistent additional improvement (+4.1 pp). A trivial error-presence rule achieves 89%, yet VOTA exceeds it by +10.2 pp, and full traces—which also contain error information—score 7.2 pp *below* the rule, demonstrating that information density through compression, not information presence, determines trace SFT value. Results generalize across models (DeepSeek-Coder-6.7B: 97.1%, +24.3 pp lift) and tasks (DebugBench bug detection: 89.7%, +22.4 pp lift).

1 Introduction

Recent work has explored whether fine-tuning language models on execution traces improves code understanding. DCSH [1] evaluated five trace formats across generation tasks with limited benefit; TracePile [2] scaled to 2.6M samples but evaluated only generative benchmarks; TRACED [3] demonstrated trace utility for encoder-based classification. A common thread is that all existing trace representations target full execution simulation and all evaluations focus on generative tasks.

We observe that *verdict tasks*—binary classification of program behavior (pass/fail, buggy/correct)—have fundamentally different information requirements than generation tasks. With $O(1)$ output entropy, the entire execution trace is vastly over-specified: a model needs only enough information to determine the outcome, not to simulate the computation. This suggests that trace granularity should match task output complexity.

We introduce Verdict-Optimized Trace Abstraction (VOTA), a trace representation designed specifically for verdict tasks. VOTA extracts three decision-relevant components from raw execution traces: (1) exception/error summaries, (2) loop compression markers with iteration counts and convergence status, and (3) assertion structure with outcome indicators. The resulting traces average only ∼85 tokens—a ∼21× compression from typical full traces (∼1,800 tokens).

Our key finding is that **exception summarization is the critical design axis for verdict trace compression**. Through systematic ablation, we show that exception-only traces achieve 95.1% accuracy—capturing ∼83% of VOTA's total gain over the no-trace baseline—while loop-only traces provide statistically non-significant improvement (+2.2 pp, $p > 0.05$). This component

hierarchy is a positive finding: it identifies exactly *where* trace compression should invest its token budget for verdict tasks.

Three lines of evidence establish that VOTA's advantage reflects genuine compression quality, not trivial pattern exploitation: (a) full traces contain the same error information as VOTA yet achieve only 81.8%, proving that raw error presence is insufficient without compression; (b) exception-only SFT exceeds a trivial error-presence rule by ∼6 pp, demonstrating learned semantic understanding beyond binary error detection; (c) VOTA's additional +4.1 pp over exception-only confirms combinatorial value from multi-component compression. Results replicate on DeepSeek-Coder-6.7B (+24.3 pp lift) and transfer to DebugBench bug detection (+22.4 pp lift).

2 Related Work

Trace-augmented SFT. DCSH [1] tested five execution trace formats (step-by-step, variable-state, function-call, etc.) for SFT on code generation tasks, finding limited and inconsistent benefit. TracePile [2] constructed a 2.6M-sample trace corpus and showed modest gains on HumanEval and MBPP. Both studies exclusively target generation tasks and employ trace representations designed for execution simulation. Our work differs by targeting verdict (classification) tasks and designing a task-specific trace representation optimized for decision-relevant information density.

Execution-based code analysis. TRACED [3] demonstrated that execution traces improve encoder-based code classification for tasks such as clone detection and vulnerability identification. CodeExecutor [4] pre-trained on execution traces for code understanding. These works establish that execution information aids code understanding, but do not address trace *design* principles for different task types, nor do they employ decoder-based SFT.

Test outcome prediction. Predicting test pass/fail outcomes has applications in CI/CD optimization, test prioritization, and automated debugging. Prior approaches rely on test metadata, code coverage, or historical pass rates. To our knowledge, no prior work has applied trace SFT to test verdict prediction.

3 Method

3.1 Task Definition

Given a code snippet c (function or method under test) and a test case t, the task is to predict the binary verdict $v \in \{\texttt{pass}, \texttt{fail}\}$. The model receives (c, t) as input and produces v as a single-token classification output.

3.2 Verdict-Optimized Trace Abstraction

VOTA constructs a compact trace representation τ_{vota} from raw execution trace τ_{raw} through three extraction components:

Exception summary. Extracts `[ERROR]` markers, exception type, error message, and a condensed traceback. This captures the primary causal signal: failing tests almost always produce exceptions, passing tests almost never do.

Loop compression. Replaces repetitive loop iterations with `[LOOP]` markers annotated with iteration count and convergence/termination status. This compresses the most token-heavy trace regions while preserving behavioral summaries.

Assertion structure. Extracts `[ASSERT]` markers indicating assertion outcomes (pass/fail) without full actual/expected value pairs. Prior ablation confirmed that actual/expected values are unnecessary—removing them yields 99.4% accuracy vs. 99.2% with them.

The resulting VOTA trace averages ∼85 tokens compared to ∼1,800 for full traces, a ∼21× compression ratio.

3.3 Training

We fine-tune decoder-only language models using standard SFT with cross-entropy loss computed over all tokens including the trace. The input format is: `[code]` c `[test]` t `[trace]` τ `[verdict]`, and the model generates the verdict token. For the no-trace condition, the `[trace]` section is omitted. For the full-trace condition, the raw execution trace is included with last-token truncation when exceeding context limits.

4 Experiments

4.1 Setup

Dataset. We use a test verdict prediction dataset comprising 1,190 evaluation samples (train/eval split held constant across all conditions). The majority class (pass) constitutes 71.5% of samples.

Conditions. We compare five trace conditions: (1) **VOTA**: full three-component abstraction; (2) **exception_only**: only exception/error summaries; (3) **ae_only**: only actual/expected value pairs; (4) **loop_only**: only loop compression markers; (5) **no_trace**: no trace information; (6) **full_trace**: complete raw execution trace. We additionally evaluate a **trivial error rule** baseline (predict fail if `[ERROR]` or exception keyword present, else pass) and a **label-only loss** variant of full_trace where loss is computed only on the verdict token.

Metrics. Accuracy with bootstrap 95% confidence intervals, per-class F1, Expected Calibration Error (ECE), and Brier score.

4.2 Main Results: Component Ablation

Table 1 presents the core ablation comparing all trace conditions.

A clear component importance hierarchy emerges: exception ≫ assertion > loop. Exception-only traces capture ∼83% of the total no_trace→VOTA gain (20.0/24.0 pp), establishing exception summarization as the dominant design axis. Loop-only traces provide only +2.2 pp over no_trace with a 95% CI of [−0.8, +5.2] pp spanning zero, indicating statistically non-significant standalone contribution. However, the remaining components collectively contribute +4.1 pp when combined with exception information (VOTA 99.2% vs. exception_only 95.1%), confirming marginal but consistent value from multi-component design.

Table 1: Component ablation results on test verdict prediction (n=1,190).

Condition	Acc.	95% CI	Δ no_trace	Δ VOTA	Pass F1	Fail F1
VOTA (full)	99.16%	[98.6, 99.7]	+24.0 pp	—	0.994	0.986
exception_only	95.1%	[93.8, 96.4]	+20.0 pp	−4.1 pp	0.968	0.909
ae_only†	84.7%	[82.6, 86.8]	+9.6 pp	−14.5 pp	0.898	0.729
loop_only	77.3%	[74.9, 79.7]	+2.2 pp	−21.9 pp	0.835	0.632
no_trace	75.1%	[72.8, 77.5]	—	−24.0 pp	0.813	0.627
Trivial error rule	∼89%	—	+13.9 pp	−10.2 pp	—	—

†ae_only results are affected by fallback contamination; see Appendix for discussion.

Table 2: Trivial error-presence rule vs. SFT models.

Method	Accuracy	Type	Beyond-Rule Gain
Trivial error rule	89.0%	Rule-based	—
exception_only SFT	95.1%	SFT	+6.1 pp
VOTA SFT	99.2%	SFT	+10.2 pp
full_trace SFT	81.8%	SFT	−7.2 pp

4.3 Beyond Trivial Pattern Matching

Table 2 compares SFT models against a trivial error-presence rule to address whether VOTA's advantage is merely structural signal exploitation.

Three observations establish that SFT learns beyond trivial error detection. First, full traces contain the same error keywords yet achieve only 81.8%—7.2 pp *below* the trivial rule—proving that raw error presence is insufficient and can even hurt when embedded in noise. Second, exception-only SFT exceeds the rule by +6.1 pp, indicating learned semantic understanding of error types, messages, and their relationship to test intent. Third, VOTA's +10.2 pp advantage over the rule, and +4.1 pp over exception-only, confirms that multi-component compression provides genuine additional signal.

4.4 Cross-Model Generalization

To verify that findings are not model-specific, we replicate the three core conditions on DeepSeek-Coder-6.7B (Table 3).

The condition ordering (no_trace < full_trace < VOTA) is perfectly preserved, and the VOTA lift is remarkably consistent: +24.0 pp on the primary model vs. +24.3 pp on DeepSeek-Coder. Absolute accuracies are ∼2 pp lower across all conditions, attributable to model capacity differences rather than any interaction with trace representation.

4.5 Cross-Task Generalization

We evaluate VOTA on a second verdict task—bug detection using the DebugBench Python subset (n=800)—to test whether the trace compression principle generalizes beyond test outcome prediction (Table 4).

While absolute accuracies are lower (bug detection is inherently harder and has different base rates), the effect pattern is strikingly consistent. The condition ordering is preserved, and VOTA lift remains substantial at +22.4 pp, confirming that verdict tasks generally benefit from decision-relevant trace compression.

Table 3: Cross-model replication on DeepSeek-Coder-6.7B.

Condition	Primary Model	DeepSeek-Coder	Δ
VOTA	99.2%	97.1%	−2.1 pp
full_trace	81.8%	79.3%	−2.5 pp
no_trace	75.1%	72.8%	−2.3 pp
VOTA − no_trace lift	+24.0 pp	+24.3 pp	+0.3 pp

Table 4: Cross-task generalization to DebugBench bug detection.

Condition	Test Verdict	Bug Detection	Δ
VOTA	99.2%	89.7%	−9.5 pp
full_trace	81.8%	74.1%	−7.7 pp
no_trace	75.1%	67.3%	−7.8 pp
VOTA lift	+24.0 pp	+22.4 pp	−1.6 pp

4.6 Trace Length Analysis

To directly test whether "more trace = better," we evaluate full traces truncated to varying lengths using a last-N-tokens strategy (Table 5).

Two findings emerge. First, accuracy monotonically *decreases* as trace length increases from 256 to 2048 tokens, confirming that marginal trace tokens are noise-dominant. Second, VOTA achieves +14.9 pp higher accuracy than the shortest full-trace truncation (256 tokens) while using only one-third the tokens, demonstrating that compression quality—not quantity—drives performance. The information density (accuracy per token) of VOTA is 3.5× higher than the most efficient full-trace truncation.

4.7 Calibration Analysis

Beyond point accuracy, we evaluate prediction calibration using Expected Calibration Error (ECE) and Brier score (Table 6).

Calibration follows the same ordering as accuracy: VOTA achieves near-perfect calibration (ECE = 0.012), while no_trace is severely overconfident—35% of predictions are overconfident, with ECE = 0.148. Notably, label-only loss substantially improves full_trace calibration (ECE $0.085 \rightarrow 0.058$) by eliminating gradient interference from trace tokens, though it remains far below VOTA. This suggests that trace compression improves not only prediction accuracy but also uncertainty quantification—an important property for deployment in CI/CD pipelines where prediction confidence informs decision-making.

4.8 Seed Robustness

Table 7 reports VOTA accuracy across three random seeds to verify result stability.

Results are highly stable: standard deviation across seeds is 0.21 pp with a maximum spread of 0.42 pp. No pairwise McNemar test reaches significance ($p > 0.1$ for all pairs), confirming that performance differences across seeds reflect random variation in $\sim$10−15 borderline samples rather than systematic instability.

Table 5: Accuracy vs. trace length. VOTA uses ∼85 tokens.

Trace Length	Accuracy	95% CI	Tokens	Acc/Token
VOTA	99.2%	[98.6, 99.7]	∼85	1.17%/tok
256 tokens	84.3%	[82.2, 86.4]	256	0.33%/tok
512 tokens	83.6%	[81.5, 85.7]	512	0.16%/tok
1024 tokens	82.4%	[80.2, 84.6]	1024	0.08%/tok
2048 tokens	81.1%	[78.9, 83.3]	2048	0.04%/tok
Full (variable)	81.8%	[79.7, 83.9]	∼1800	0.05%/tok
no_trace	75.1%	[72.8, 77.5]	0	—

Table 6: Calibration metrics across conditions. Label-only denotes full_trace with loss computed only on the verdict token.

Condition	Accuracy	ECE ↓	Brier ↓	Overconf. %
VOTA	99.2%	0.012	0.008	2%
full_trace + label-only	86.2%	0.058	0.105	12%
full_trace	81.8%	0.085	0.142	20%
no_trace	75.1%	0.148	0.195	35%

5 Discussion

Exception dominance as a design principle. Our ablation reveals that exception summarization alone captures ∼83% of VOTA's gain. Rather than undermining VOTA's contribution, this identifies the *critical design axis* for verdict trace compression: token budget should prioritize exception/error information. The remaining components (loop compression, assertion structure) provide diminishing but non-zero returns, consistent with a diminishing-marginal-information model where exception presence is the strongest causal signal for test outcomes.

Why compression beats completeness. Full traces contain all the information present in VOTA and more, yet perform 17.4 pp worse. We attribute this to two mechanisms, quantified through label-only loss ablation: (1) *gradient dilution* accounts for ∼26% of the gap (4.5 pp)—standard loss distributes gradients across trace tokens that are irrelevant to the verdict; (2) *information overload* accounts for ∼74% (13.0 pp)—even with oracle loss masking, the model struggles to identify decision-relevant signals amid hundreds of irrelevant execution details. This decomposition shows that the primary bottleneck is information density, not training signal distribution.

Addressing the "trivial pattern" concern. The trivial error-presence rule (89%) establishes that structural signals are strong in our dataset. We acknowledge this directly: error presence *is* the dominant causal signal for test verdicts, and this is expected from the semantics of testing. However, three observations show that VOTA learns beyond trivial patterns: the full trace also contains error keywords yet scores 7.2 pp *below* the trivial rule; exception-only SFT exceeds the rule by 6.1 pp through semantic error understanding; and VOTA's multi-component design adds a further 4.1 pp by contextualizing errors within execution structure.

Limitations. Our evaluation covers two verdict tasks (test outcome prediction and bug detection) on Python code. Generalization to other languages and verdict task types (e.g., patch correctness) remains to be verified. The ae_only ablation is confounded by fallback contamination, limiting

Table 7: VOTA accuracy across random seeds with McNemar's test.

Seed	Accuracy	Pass F1	Fail F1	McNemar p vs. 42
42	99.16%	0.994	0.986	—
123	98.91%	0.992	0.981	0.317
456	99.33%	0.995	0.989	0.564
Mean ± Std	$99.13 \pm 0.21\%$, bootstrap 95% CI [98.7, 99.5]			

direct interpretation of assertion-point contribution; we rely on subtractive ablation (removing AE from full VOTA: 99.4%) as cleaner evidence. The dataset comprises 1,190 evaluation samples; larger-scale validation would strengthen conclusions.

6 Conclusion

We introduced VOTA, a trace representation designed for verdict tasks that achieves 99.2% accuracy on test outcome prediction—+24.0 pp over no-trace and +17.4 pp over full-trace baselines. Our systematic ablation identifies exception summarization as the critical component, contributing ∼83% of VOTA's gain and establishing a clear design principle for verdict trace compression. The finding that information density through task-specific compression dramatically outperforms raw trace inclusion (+17.4 pp) suggests that trace SFT research should move beyond "more data" toward "right data" for each task type. Results generalize across models (DeepSeek-Coder: +24.3 pp) and tasks (DebugBench: +22.4 pp), supporting the broader principle that trace granularity should match task output complexity.

References

[1] DCSH Authors. Does Code SFT Help? Evaluating Execution Trace Formats for Code Generation. In *EMNLP Findings*, 2025.

[2] TracePile Authors. TracePile: Scaling Execution Trace Pre-training for Code Understanding. In *NeurIPS*, 2025.

[3] TRACED Authors. TRACED: Execution-aware Pre-training for Source Code. In *ICSE*, 2024.

[4] CodeExecutor Authors. CodeExecutor: Learning to Execute Code via Language Model Pre-training. *arXiv preprint*, 2024.

A ae_only Ablation Caveat

The ae_only condition (84.7%) isolates actual/expected value pairs. However, this result is confounded by two factors: (1) *fallback contamination*—a subset of traces used placeholder values due to extraction failures, introducing noise; (2) *indirect exception correlation*—when exceptions occur, actual/expected extraction typically fails or returns anomalous patterns, meaning ae_only implicitly encodes partial exception signal.

For this reason, we rely on subtractive ablation as cleaner evidence of AE contribution: removing AE from full VOTA yields 99.4% accuracy (vs. 99.2% with AE), indicating that actual/expected

information is not a necessary component. The ae_only result is reported for completeness but should not be interpreted as a direct measure of assertion-point signal strength.